

OWASP For Web - Lab 2

- Due Saturday by 11:59pm
- Points 500
- Submitting a text entry box or a file upload

OWASP For Web - Lab 2

This OWASP lab series cover the **most critical security risks** across modern cyber attacks against: **web apps, APIs, and LLM/GenAI applications**. They complement each other: web apps remain the user-facing core, APIs expose business logic and data to services and partners, and LLMs/agents introduce new attack surfaces and failure modes. Together, they provide a **defense-in-depth view of application security** grounded in OWASP's community-curated Top 10 lists and guidance.

a) Overview and the topics in this lab

This lab series builds core web application secure coding fundamentals. The skills here are the baseline you need even when working on AI systems, because most LLM applications are still delivered through web apps, dashboards, admin consoles, and portals. Here is an overview of the all areas covered:

1. **Clickjacking**: Tricking a user into clicking an invisible button or link overlaying a legitimate page to trigger unauthorized actions.
2. **Command Injection**: Injecting arbitrary operating system commands into a vulnerable application input to execute them on the host server.
3. **Components with Known Vulnerabilities**: Deploying libraries or frameworks with unpatched CVEs (e.g., outdated jQuery or Struts) that attackers can easily exploit.
4. **Cross Site Request Forgery (CSRF)**: Forcing an authenticated user's browser to send an unwanted request to a web application where they are currently logged in.
5. **Directory Traversal**: Manipulating file paths (e.g., ../../) to access restricted directories and files outside the web root folder.
6. **DOM XSS**: Executing malicious JavaScript by modifying the Document Object Model (DOM) environment in the victim's browser.
7. **Forced Browsing**: Accessing restricted pages or resources by guessing their URLs (e.g., /admin) rather than following legitimate links.
8. **Header Injection**: Injecting malicious characters (like CRLF) into HTTP headers to split responses, hijack sessions, or poison caches.
9. **Horizontal Privilege Escalation**: Accessing the private data or account functions of another user with the same access level (e.g., User A viewing User B's profile).
10. **Insecure URL Redirect**: Redirecting users to an untrusted external site via unvalidated URL parameters, often used for phishing.

11. **Leftover Debug Code:** Exposing sensitive debugging endpoints or comments in production code that reveal backend logic or credentials.
12. **Log4j 2 (CVE-2021-44228):** Exploiting a critical RCE vulnerability in the Log4j logging library via JNDI lookup injection.
13. **PII data in URL:** Leaking Personally Identifiable Information (like emails or tokens) by passing them in URL parameters which are logged by servers.
14. **Reflected XSS:** Injecting malicious scripts that are immediately returned (reflected) by the server in an error message or search result.
15. **Ruby rest-client Backdoor:** A specific supply chain attack where a compromised version of a popular Ruby gem contained malicious code to steal credentials.
16. **Server Side Request Forgery (SSRF):** Coercing the server to make HTTP requests to internal resources or metadata services that should be inaccessible.
17. **Session Fixation:** Forcing a user to use a known session ID so the attacker can hijack the session once the user logs in.
18. **SQL Injection:** Inserting malicious SQL queries into input fields to manipulate the backend database and retrieve or delete data.
19. **Stored Cross Site Scripting:** Permanently storing malicious scripts on the server (e.g., in a comment) that execute whenever a victim views the page.
20. **TikTok Cross Site Scripting:** A real-world case study demonstrating how a specific XSS vulnerability in TikTok's web infrastructure was exploited.
21. **Token Exposure in URL:** Passing authentication or reset tokens in the URL bar where they can be leaked via browser history or referrer headers.
22. **User Enumeration:** Determining valid usernames by analyzing differences in error messages or response times during login attempts.
23. **Vertical Privilege Escalation:** A lower-privileged user (e.g., 'guest') exploiting a flaw to gain access to higher-privileged functions (e.g., 'admin').
24. **Weak Randomness:** Using predictable algorithms for generating security tokens or passwords, making them easy for attackers to guess.
25. **XML Injection:** Injecting malicious XML tags or entities to manipulate XML processing logic or access sensitive files (XXE).

b) NotebookLM learning approach, with sample guidance for each of the 25

Use NotebookLM as a structured reviewer after each exercise:

- Ask for the vulnerability's root cause.
- Ask what a secure coding fix looks like in the language or framework you know.
- Ask for a small test plan you could use to prove the fix works.

Sample guidance per topic:

1. **Clickjacking:** How do the X-Frame-Options and Content-Security-Policy(frame-ancestors) headers technically prevent a page from being loaded inside a malicious iframe?
2. **Command Injection:** Explain the difference between **Command Injection**(executing OS commands) and **Code Injection**(executing application code), and provide a secure code example using `execFile` instead of `exec` in Node.js.
3. **Vulnerable Components:** What is a Software Bill of Materials (SBOM), and how does Software Composition Analysis (SCA) help mitigate the risk of using libraries with known CVEs?
4. **CSRF:** Why is the **Synchronizer Token Pattern**(Anti-CSRF Token) effective against Cross-Site Request Forgery, and how does the SameSite cookie attribute act as a defense-in-depth measure?
5. **Directory Traversal:** How can an attacker use `../` sequences to access files outside the web root, and why is validating user input against an **allowlist** of filenames safer than trying to filter out dot-dot-slash sequences?
6. **DOM XSS:** Explain the concept of **Sources** and **Sinks** in the Document Object Model (DOM). How does DOM-based XSS differ from Reflected XSS in terms of where the payload is executed?
7. **Forced Browsing:** What is the difference between **Security through Obscurity**(hiding a URL) and proper **Access Control**, and how can fuzzing tools discover hidden endpoints?
8. **Header Injection:** How does **CRLF Injection**(Carriage Return Line Feed) allow an attacker to inject malicious HTTP headers or split the HTTP response to poison the cache?
9. **Horizontal Privilege Escalation:** Explain how **Insecure Direct Object References (IDOR)** lead to horizontal privilege escalation, where User A can access User B's data by simply changing an ID in the URL.
10. **Insecure URL Redirect:** How can an Open Redirect vulnerability be used to facilitate phishing attacks, and what is the secure way to implement redirects using a server-side map instead of user input?
11. **Leftover Debug Code:** What are the security risks of leaving debug endpoints or verbose error logging enabled in a production environment, and how can this aid an attacker in reconnaissance?
12. **Log4j 2 (Log4Shell):** Explain the mechanism of **JNDI Injection** in the Log4j vulnerability (CVE-2021-44228). How did the `${jndi:ldap://...}` payload allow remote code execution?
13. **PII data in URL:** Why is passing PII (like emails or tokens) in URL parameters considered a vulnerability even if HTTPS is used? (Hint: Consider browser history, proxy logs, and the Referer header).
14. **Reflected XSS:** What is **Context-Aware Output Encoding**, and why is it the primary defense against Reflected Cross-Site Scripting?
15. **Ruby rest-client Backdoor:** Using the Ruby rest-client incident as a case study, explain how a **Supply Chain Attack** works and why developers should pin dependency versions.

16. **SSRF**: How does **Server-Side Request Forgery (SSRF)** allow an attacker to scan internal networks or access cloud metadata services (like AWS 169.254.169.254)?
17. **Session Fixation**: What is **Session Fixation**, and why must the application invalidate the existing session ID and issue a new one immediately after a user authenticates?
18. **SQL Injection**: Show me a code example comparing a vulnerable SQL query using string concatenation versus a secure query using **Prepared Statements (Parameterized Queries)**.
19. **Stored XSS**: Why is **Stored XSS** (Persistent XSS) considered more critical than Reflected XSS, and how does Content Security Policy (CSP) help mitigate the impact if injection occurs?
20. **TikTok XSS**: In the TikTok XSS case study, how did attackers chain a reflected XSS vulnerability with other flaws to achieve account takeover or data exfiltration?
21. **Token Exposure in URL**: Explain how the **HTTP Referer header** can accidentally leak sensitive reset tokens to third-party analytics sites if the tokens are placed in the URL.
22. **User Enumeration**: How do verbose error messages (e.g., 'User not found' vs. 'Incorrect password') and **Timing Attacks** allow attackers to build a list of valid usernames?
23. **Vertical Privilege Escalation**: What is **Role-Based Access Control (RBAC)**, and why must authorization checks be performed on **every** request on the server side, not just hidden in the UI?
24. **Weak Randomness**: What is the difference between a **Pseudo-Random Number Generator (PRNG)** and a **Cryptographically Secure PRNG (CSPRNG)**, and why is `Math.random()` unsafe for generating security tokens?
25. **XML Injection**: How does **XML External Entity (XXE)** injection allow attackers to read local files (like `/etc/passwd`), and what specific configuration must be disabled in the XML parser to prevent it?

NotebookLM link: <https://notebooklm.google.com>

c) Submission requirements

You pick **13** out of 25 and submit **13 screenshots and 13 paragraphs** for this lab.

Screenshots

Provide 1 “final state” screenshot per topic showing the completed module or the final demonstrated outcome (for example the exploited behavior, plus the secure behavior after mitigation if the lab includes a defend phase). Use consistent naming:

Reflection paragraphs

Provide **13** paragraphs, one per topic

Each paragraph should include:

- What the risk means in your own words
- What you did or observed in the lab scenario

Lab link: <https://application.security/free/owasp-top-10>

Have fun and please use student lounge for second opinion or questions!

File Upload

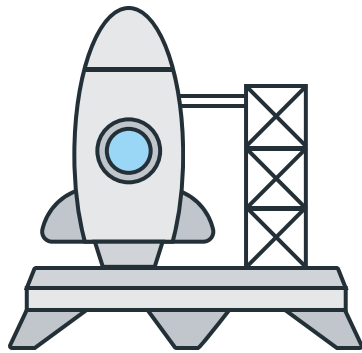
[Text Entry](#)

[Dropbox](#)

[JoVE LTI Tool](#)

[Microsoft Education](#)

Upload a file, or choose a file you've already uploaded.



Drag a file here, or

[Choose a file to upload](#)



Use Webcam

+ [Add Another File](#)

[Click here to find a file you've already uploaded](#)

Comments...

Cancel

Submit Assignment